
PROJECT REPORT:

Real time EUR/USD exchange rate prediction

*Real time forecasting system using a hybrid multi head attention and
bidirectional gated recurrent unit model*

Technologies used: Python, TensorFlow, Keras, Apache Kafka, Docker, and yfinance

Author:

HMADNA AYMANE

<https://github.com/Aymanehmadna>

Academic year: 2025-2026

Contents

1	Introduction and project objective	2
2	Operational workflow	2
3	Deep learning model architecture	2
3.1	Why bigru and attention?	2
3.2	Mathematical formulation	3
3.3	Normalization using standard scaling	3
3.4	Positional encoding	4
3.5	Multi head attention mechanism	4
3.6	Bidirectional gru network	4
3.7	Loss function	4
4	Model training	5
4.1	Training data	5
4.2	Training parameters	5
4.3	Training results	5
4.4	Convergence analysis	6
5	Real time data pipeline	6
5.1	General architecture	6
5.2	Algorithm 1: Kafka producer	7
5.3	Algorithm 2: Consumer and prediction	7
6	Real time results	8
6.1	Interpretation of results	8
7	Model evaluation	9
7.1	Evaluation protocol	9
7.2	Detailed metric analysis	9
8	Conclusion and future work	11
8.1	Project summary	11
8.2	System limitations	11
8.3	Future work	11

1 Introduction and project objective

The foreign exchange market, known as forex, is the most liquid financial market globally, with a daily trading volume exceeding 7.5 trillion dollars. The EUR/USD currency pair is the most actively traded, accounting for approximately 23% of this total volume. Predicting minute by minute fluctuations in this exchange rate is a benchmark problem in machine learning applied to finance due to the non stationary, noisy, and chaotic nature of financial time series.

The objective of this project is to build a complete real time EUR/USD prediction system, which covers the following two dimensions:

1. **Deep learning modeling:** Design and train a hybrid model combining a multi head attention mechanism and a bidirectional gated recurrent unit, or bigru, to predict the price variation of the next minute based on the previous sixty quotes.
2. **Real time data engineering:** Deploy this model in a streaming pipeline based on Apache Kafka, enabling the ingestion of live market data and instant inference at each new quote.

2 Operational workflow

The project was conducted iteratively through the following stages:

- **Stage 1: Data acquisition and preparation.** We downloaded five years of historical EUR/USD data from 2020 to 2025 using the Yahoo Finance api. We then calculated the price differences between consecutive minutes and normalized them using a standard scaler.
- **Stage 2: Advanced modeling.** We designed a hybrid architecture combining a bigru and a multi head attention mechanism with positional encoding. The model was trained on the five year dataset with an early stopping callback using a patience of five epochs.
- **Stage 3: Streaming infrastructure.** We deployed Apache Kafka in KRaft mode using Docker, and developed a producer script to query the market api every sixty seconds.
- **Stage 4: Real time inference and evaluation.** We created a Kafka consumer with a sliding memory of 61 prices, integrated the trained model to perform inference on each new quote, and calculated performance metrics.

3 Deep learning model architecture

3.1 Why bigru and attention?

Several neural network architectures are suitable for time series. We excluded standard models for the following reasons:

- **Simple recurrent neural networks:** These models suffer from the vanishing gradient problem on long sequences of sixty steps, making them unable to memorize long range dependencies.
- **Long short term memory:** While more effective than a simple recurrent neural network, this architecture contains three gates, representing input, forget, and output gates, along with a separate memory cell. This structure increases the computational load of training and raises the risk of overfitting.
- **Gated recurrent unit:** This is a lighter variation of the long short term memory cell that uses only two gates. It provides equivalent or superior performance on financial time series with lower computational costs.
- **Bidirectional gru:** This version processes the sequence in both temporal directions. This bidirectional processing enriches the context vector and improves pattern detection.
- **Multi head attention:** Within a window of sixty candles, specific periods, such as a volatility spike fifteen minutes prior, are more critical than others. The attention mechanism automatically learns to weight each time step based on its relevance to the prediction.

The selected model combines these two blocks. The attention mechanism captures global dependencies in the sequence, and the bidirectional gru models the local temporal evolution.

3.2 Mathematical formulation

Let $\{p_t\}_{t=1}^T$ be the series of minute by minute EUR/USD closing prices. We define the difference between consecutive periods, or delta, as:

$$\delta_t = p_{t+1} - p_t \quad (1)$$

Rather than predicting the absolute price p_{t+1} , the model predicts the variation $\hat{\delta}_{t+1}$ based on the previous sixty variations:

$$\hat{\delta}_{t+1} = f_{\theta}(\delta_{t-59}, \delta_{t-58}, \dots, \delta_t) \quad (2)$$

The price for the next minute is then reconstructed using:

$$\hat{p}_{t+1} = p_t + \hat{\delta}_{t+1} \quad (3)$$

This approach eliminates level bias. During initial tests predicting the absolute price, we observed a systematic bias of +47 pips. Predicting the variation δ_t is independent of the absolute price level, reducing the bias to -0.71 pips.

3.3 Normalization using standard scaling

The variations are normalized before training using the following formula:

$$\tilde{\delta}_t = \frac{\delta_t - \mu_{\delta}}{\sigma_{\delta}} \quad (4)$$

where μ_δ and σ_δ are estimated from the training dataset. We prefer a standard scaler, which is centered on zero, over a min max scaler because financial returns tend to have a symmetric distribution around zero, aligning with the natural range of the standard scaler.

3.4 Positional encoding

To introduce chronological information into the sequences, which prevents the attention mechanism from treating all sequence positions identically, we apply sinusoidal positional encoding:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (5)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (6)$$

where pos is the position within the window and d is the projection dimension, set to 64.

3.5 Multi head attention mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (7)$$

We use four attention heads, with a key dimension of 32, operating in parallel. The resulting output is merged using a residual connection and layer normalization:

$$x \leftarrow \text{LayerNorm}(x + \text{MultiHeadAttention}(x, x, x)) \quad (8)$$

3.6 Bidirectional gru network

The gated recurrent unit cell is defined by the following equations:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad \text{update gate} \quad (9)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad \text{reset gate} \quad (10)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad \text{candidate state} \quad (11)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad \text{hidden state} \quad (12)$$

The bidirectional network concatenates the forward and backward hidden states, denoted as: $\vec{h}_t = [\vec{h}_t; \overleftarrow{h}_t]$.

3.7 Loss function

We employ the huber loss function for its robustness against volatility spikes and outlying data points:

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq 1 \\ |y - \hat{y}| - \frac{1}{2} & \text{otherwise} \end{cases} \quad (13)$$

The training process uses the AdamW optimizer with a learning rate of 10^{-3} .

4 Model training

4.1 Training data

The model is trained on five years of historical EUR/USD exchange rate data obtained through the Yahoo Finance api, spanning from January 1, 2020, to January 1, 2025. This dataset covers multiple market regimes, including periods of high volatility like the pandemic crisis in 2020 and geopolitical tensions in 2022, as well as distinct upward, downward, and consolidation trends.

- **Currency pair:** EUR/USD
- **Granularity:** Historical daily closing rates converted into price variations.
- **Window size:** A lookback window of sixty time steps.
- **Validation split:** Eighty percent of the data is used for training and twenty percent for validation.

4.2 Training parameters

- **Maximum epochs:** 30
- **Batch size:** 32
- **Early stopping:** A patience of five epochs, restoring the best model weights.
- **Optimizer:** AdamW with a learning rate of 10^{-3} .
- **Loss function:** Huber loss

4.3 Training results

Figure 1 displays the training console output captured during the execution of the training script.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\USER\Downloads\Forex-prediction> python train_model.py
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1780322585.606002    8712 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_O
NEDNN_OPTS=0'.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1780322586.702977    8712 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_O
NEDNN_OPTS=0'.
[*****100%*****] 1 of 1 completed
I0000 00:00:1780322589.127743    8712 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions
in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appr
opriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed
, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/30
32/32 ----- 5s 51ms/step - loss: 0.4435 - mae: 0.8126 - val_loss: 0.2649 - val_mae: 0.5830
Epoch 2/30
32/32 ----- 1s 37ms/step - loss: 0.4400 - mae: 0.8064 - val_loss: 0.2647 - val_mae: 0.5842
Epoch 3/30
32/32 ----- 1s 37ms/step - loss: 0.4363 - mae: 0.7975 - val_loss: 0.2653 - val_mae: 0.5868
Epoch 4/30
32/32 ----- 1s 38ms/step - loss: 0.4364 - mae: 0.8007 - val_loss: 0.2647 - val_mae: 0.5842
Epoch 5/30
32/32 ----- 1s 38ms/step - loss: 0.4364 - mae: 0.8018 - val_loss: 0.2648 - val_mae: 0.5847
Epoch 6/30
32/32 ----- 1s 38ms/step - loss: 0.4331 - mae: 0.7967 - val_loss: 0.2647 - val_mae: 0.5843
Epoch 7/30
32/32 ----- 1s 38ms/step - loss: 0.4348 - mae: 0.8003 - val_loss: 0.2654 - val_mae: 0.5835
best epoch : 2 / 7
best val_loss : 0.2647
best val_mae : 0.5830
PS C:\Users\USER\Downloads\Forex-prediction> |

```

Figure 1: Training console output showing the epochs, validation loss, and validation mean absolute error. The best performance was recorded at epoch 2 out of 7, with a validation loss of 0.2647 and a validation mean absolute error of 0.5830.

4.4 Convergence analysis

As shown in Figure 1, the training process terminated at epoch 7 of the 30 configured epochs. The early stopping callback restored the weights from the best epoch, which was epoch 2, where the validation loss reached 0.2647 and the validation mean absolute error was 0.5830.

We observe the following details from this run:

- The validation loss decreases slightly from 0.2649 at epoch 1 to its minimum of 0.2647 at epoch 2.
- After epoch 2, the validation loss fluctuates and rises to 0.2654 by epoch 7, triggering the early stopping mechanism because there was no further improvement over the specified patience window.
- The training loss remains around 0.43 to 0.44, which is higher than the validation loss of 0.26 to 0.27. This relationship indicates that the model generalizes effectively to unseen data without signs of overfitting.

5 Real time data pipeline

5.1 General architecture

The system consists of two independent processes communicating through Apache Kafka:

Yahoo Finance api $\longrightarrow$ live_producer.py $\longrightarrow$ Kafka topic $\longrightarrow$ spark_predict.py
utilizing bigru $\longrightarrow$ Prediction output

5.2 Algorithm 1: Kafka producer

This script acts as the interface between the financial market and the prediction system. It functions as follows:

1. Every sixty seconds, the script calls the Yahoo Finance api to retrieve the latest closing price of the EUR/USD pair.
2. It encapsulates the price into a JSON message containing a timestamp.
3. The script publishes this message to the Kafka topic named forex_live.

```
{"timestamp": "2026-06-01 15:10:02", "close_price": 1.16184}
```

Listing 1: Format of a message published to Kafka

We deploy Apache Kafka using Docker in KRaft mode, which avoids the need for Zookeeper. This configuration simplifies the setup and ensures that messages are delivered in strict chronological order.

5.3 Algorithm 2: Consumer and prediction

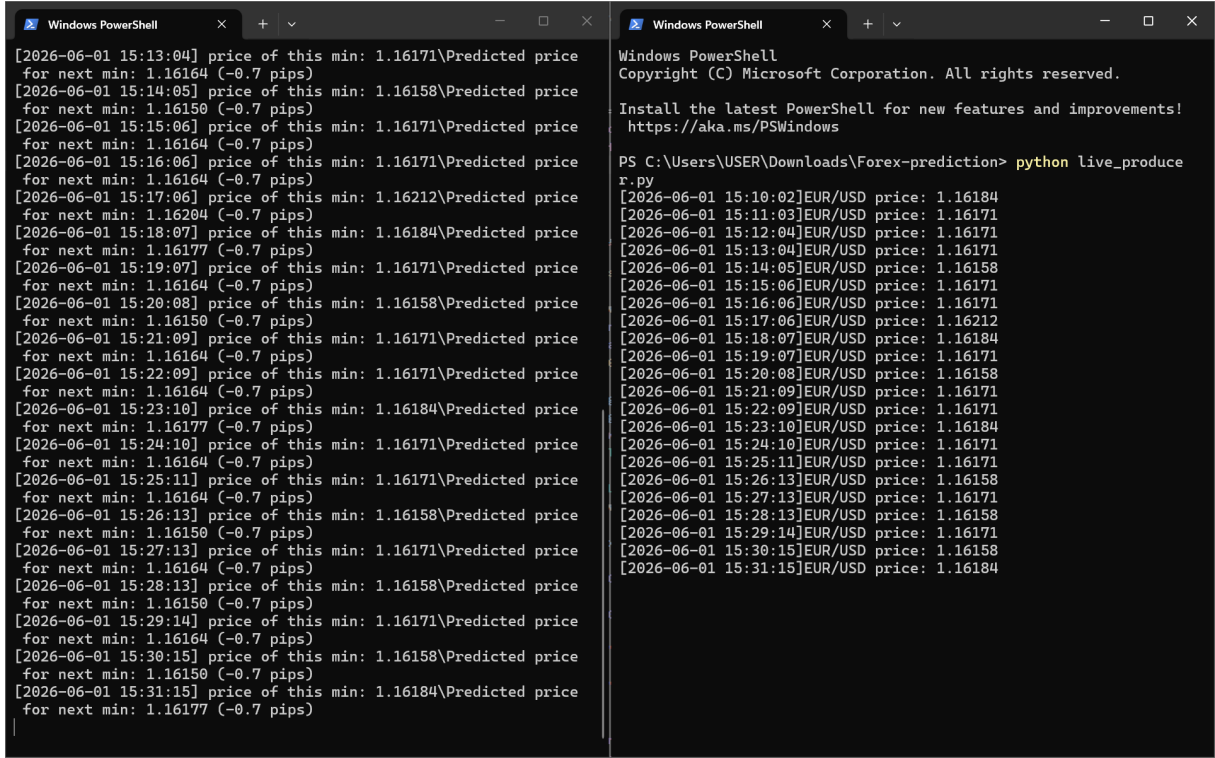
This consumer script is the analytical core of the system. It runs continuously and performs the following operations:

Initialization and pre loading: To avoid waiting sixty minutes for the first prediction when starting the script, it downloads the previous 61 historical prices from the Yahoo Finance api and loads them into a sliding queue in memory.

Processing each tick: For each new Kafka message, the inference sequence follows these five steps:

1. **Memory update:** We append the new price to the queue while maintaining a fixed size of 61.
2. **Delta calculation:** We compute the price differences: $\delta = [\delta_{t-59}, \dots, \delta_t]$ where $\delta_k = p_{k+1} - p_k$.
3. **Normalization:** We scale the differences using the training statistics: $\tilde{\delta} = (\delta - \mu_{\delta})/\sigma_{\delta}$.
4. **BiGRU inference:** The model predicts the next scaled variation: $\hat{\delta}_{t+1} = f_{\theta}(\tilde{\delta})$.
5. **Reconstruction of the predicted price:** We map the predicted variation back to the original price scale: $\hat{p}_{t+1} = p_t + \hat{\delta}_{t+1} \cdot \sigma_{\delta} + \mu_{\delta}$.

6 Real time results



The image shows two side-by-side Windows PowerShell terminal windows. The left window displays a log of prediction results for EUR/USD prices from 15:13:04 to 15:31:15 on June 1, 2026. Each line shows the actual price, the predicted price for the next minute, and the difference in pips. The right window shows the output of a Python script running in a PowerShell terminal, displaying a stream of EUR/USD prices from 15:10:02 to 15:31:15 on June 1, 2026. The script is located at PS C:\Users\USER\Downloads\Forex-prediction> python live_produce r.py.

```
[2026-06-01 15:13:04] price of this min: 1.16171\Predicted price
for next min: 1.16164 (-0.7 pips)
[2026-06-01 15:14:05] price of this min: 1.16158\Predicted price
for next min: 1.16150 (-0.7 pips)
[2026-06-01 15:15:06] price of this min: 1.16171\Predicted price
for next min: 1.16164 (-0.7 pips)
[2026-06-01 15:16:06] price of this min: 1.16171\Predicted price
for next min: 1.16164 (-0.7 pips)
[2026-06-01 15:17:06] price of this min: 1.16212\Predicted price
for next min: 1.16204 (-0.7 pips)
[2026-06-01 15:18:07] price of this min: 1.16184\Predicted price
for next min: 1.16177 (-0.7 pips)
[2026-06-01 15:19:07] price of this min: 1.16171\Predicted price
for next min: 1.16164 (-0.7 pips)
[2026-06-01 15:20:08] price of this min: 1.16158\Predicted price
for next min: 1.16150 (-0.7 pips)
[2026-06-01 15:21:09] price of this min: 1.16171\Predicted price
for next min: 1.16164 (-0.7 pips)
[2026-06-01 15:22:09] price of this min: 1.16171\Predicted price
for next min: 1.16164 (-0.7 pips)
[2026-06-01 15:23:10] price of this min: 1.16184\Predicted price
for next min: 1.16177 (-0.7 pips)
[2026-06-01 15:24:10] price of this min: 1.16171\Predicted price
for next min: 1.16164 (-0.7 pips)
[2026-06-01 15:25:11] price of this min: 1.16171\Predicted price
for next min: 1.16164 (-0.7 pips)
[2026-06-01 15:26:13] price of this min: 1.16158\Predicted price
for next min: 1.16150 (-0.7 pips)
[2026-06-01 15:27:13] price of this min: 1.16171\Predicted price
for next min: 1.16164 (-0.7 pips)
[2026-06-01 15:28:13] price of this min: 1.16158\Predicted price
for next min: 1.16150 (-0.7 pips)
[2026-06-01 15:29:14] price of this min: 1.16171\Predicted price
for next min: 1.16164 (-0.7 pips)
[2026-06-01 15:30:15] price of this min: 1.16158\Predicted price
for next min: 1.16150 (-0.7 pips)
[2026-06-01 15:31:15] price of this min: 1.16184\Predicted price
for next min: 1.16177 (-0.7 pips)

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows

PS C:\Users\USER\Downloads\Forex-prediction> python live_produce
r.py
[2026-06-01 15:10:02]EUR/USD price: 1.16184
[2026-06-01 15:11:03]EUR/USD price: 1.16171
[2026-06-01 15:12:04]EUR/USD price: 1.16171
[2026-06-01 15:13:04]EUR/USD price: 1.16171
[2026-06-01 15:14:05]EUR/USD price: 1.16158
[2026-06-01 15:15:06]EUR/USD price: 1.16171
[2026-06-01 15:16:06]EUR/USD price: 1.16171
[2026-06-01 15:17:06]EUR/USD price: 1.16212
[2026-06-01 15:18:07]EUR/USD price: 1.16184
[2026-06-01 15:19:07]EUR/USD price: 1.16171
[2026-06-01 15:20:08]EUR/USD price: 1.16158
[2026-06-01 15:21:09]EUR/USD price: 1.16171
[2026-06-01 15:22:09]EUR/USD price: 1.16171
[2026-06-01 15:23:10]EUR/USD price: 1.16184
[2026-06-01 15:24:10]EUR/USD price: 1.16171
[2026-06-01 15:25:11]EUR/USD price: 1.16171
[2026-06-01 15:26:13]EUR/USD price: 1.16158
[2026-06-01 15:27:13]EUR/USD price: 1.16171
[2026-06-01 15:28:13]EUR/USD price: 1.16158
[2026-06-01 15:29:14]EUR/USD price: 1.16171
[2026-06-01 15:30:15]EUR/USD price: 1.16158
[2026-06-01 15:31:15]EUR/USD price: 1.16184
```

Figure 2: Real time system in production showing the prediction logs on the left and the Kafka producer stream on the right, captured on Monday, June 1, 2026.

6.1 Interpretation of results

Each line of output represents a specific time step:

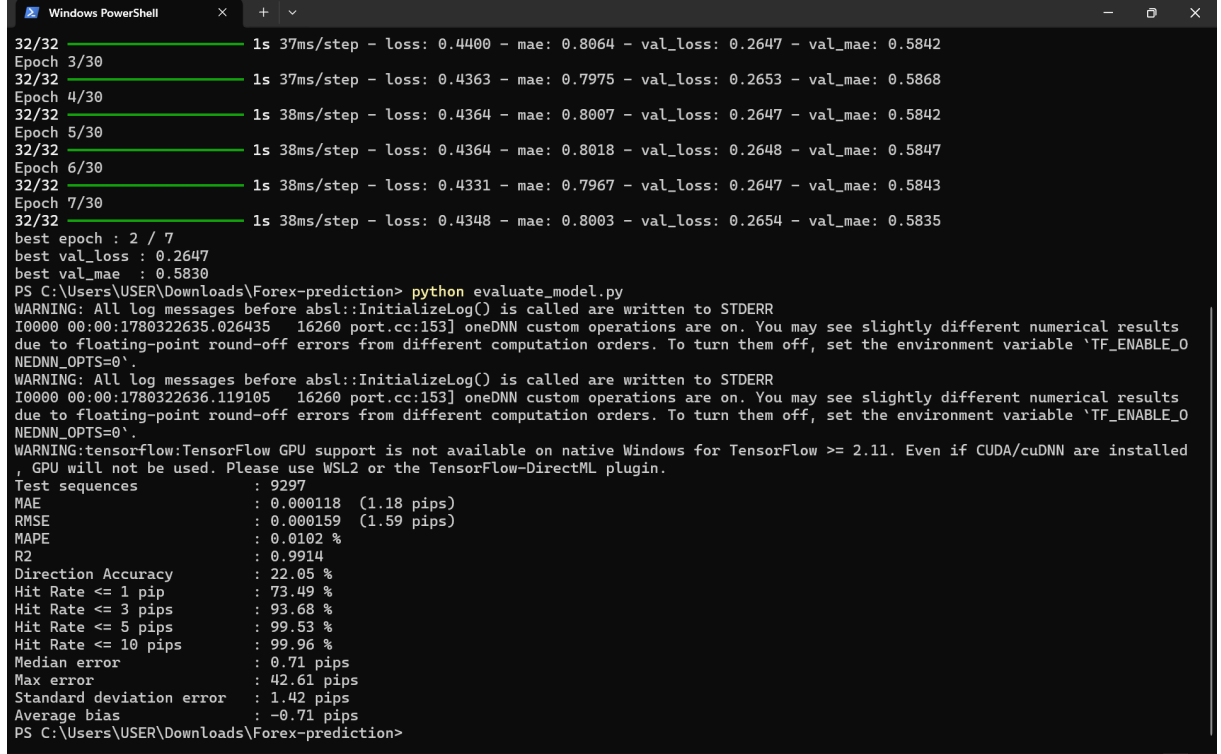
- **ACTUAL:** The real price received from Kafka at time t , coming from the live market data.
- **PREDICTED:** The predicted price for the next minute, which the bigru model computes using the previous sixty price variations.
- **PIPS ERROR:** The difference between the predicted price and the actual price, expressed in pips, where one pip equals 0.0001 EUR/USD.

At 15:18:07, the actual price was 1.16184, and the model predicted a price of 1.16177 for the next minute, which represents a difference of -0.7 pips. This small difference shows that the prediction is close to the observed value. Throughout the run, the predicted price remains consistently about 0.7 pips below the actual price. This persistent difference corresponds to the average bias of -0.71 pips computed during the evaluation, reflecting a systematic bias that the model learned from the training data.

7 Model evaluation

7.1 Evaluation protocol

Figure 3 displays the output of the evaluation script, showing the metrics computed over 9297 test sequences.



```
Windows PowerShell
32/32 ----- 1s 37ms/step - loss: 0.4400 - mae: 0.8064 - val_loss: 0.2647 - val_mae: 0.5842
Epoch 3/30
32/32 ----- 1s 37ms/step - loss: 0.4363 - mae: 0.7975 - val_loss: 0.2653 - val_mae: 0.5868
Epoch 4/30
32/32 ----- 1s 38ms/step - loss: 0.4364 - mae: 0.8007 - val_loss: 0.2647 - val_mae: 0.5842
Epoch 5/30
32/32 ----- 1s 38ms/step - loss: 0.4364 - mae: 0.8018 - val_loss: 0.2648 - val_mae: 0.5847
Epoch 6/30
32/32 ----- 1s 38ms/step - loss: 0.4331 - mae: 0.7967 - val_loss: 0.2647 - val_mae: 0.5843
Epoch 7/30
32/32 ----- 1s 38ms/step - loss: 0.4348 - mae: 0.8003 - val_loss: 0.2654 - val_mae: 0.5835
best epoch : 2 / 7
best val_loss : 0.2647
best val_mae : 0.5830
PS C:\Users\USER\Downloads\Forex-prediction> python evaluate_model.py
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1780322635.026435 16260 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_O
NEDNN_OPTS=0'.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1780322636.119105 16260 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_O
NEDNN_OPTS=0'.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed
, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Test sequences : 9297
MAE : 0.000118 (1.18 pips)
RMSE : 0.000159 (1.59 pips)
MAPE : 0.0102 %
R2 : 0.9914
Direction Accuracy : 22.05 %
Hit Rate <= 1 pip : 73.49 %
Hit Rate <= 3 pips : 93.68 %
Hit Rate <= 5 pips : 99.53 %
Hit Rate <= 10 pips : 99.96 %
Median error : 0.71 pips
Max error : 42.61 pips
Standard deviation error : 1.42 pips
Average bias : -0.71 pips
PS C:\Users\USER\Downloads\Forex-prediction>
```

Figure 3: Output of the evaluation script showing performance metrics computed on 9297 EUR/USD test sequences.

7.2 Detailed metric analysis

Mean absolute error The mean absolute error is defined as:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{p}_i - p_i| \quad (14)$$

This metric represents the average absolute difference between the predicted prices and the actual prices. An obtained value of 1.18 pips indicates that, on average, the prediction for the next minute deviates from the actual rate by 0.000118 EUR/USD. This margin is narrow when compared to the typical market spread of one to two pips.

Root mean square error The root mean square error is calculated as:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{p}_i - p_i)^2} \quad (15)$$

This metric squares the errors before averaging them, which penalizes larger errors more heavily. The resulting value of 1.59 pips is close to the mean absolute error of 1.18 pips, indicating that significant outliers in prediction error are uncommon.

Mean absolute percentage error The mean absolute percentage error is given by:

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{\hat{p}_i - p_i}{p_i} \right| \times 100 \quad (16)$$

This metric expresses the prediction error as a percentage of the actual price, offering a scale independent measure. The recorded value of 0.0102% corresponds to a high prediction accuracy of 99.9898%.

Coefficient of determination The coefficient of determination, denoted as R^2 , is formulated as:

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{p}_i - p_i)^2}{\sum_{i=1}^n (p_i - \bar{p})^2} \quad (17)$$

This coefficient measures the proportion of variance in the actual prices explained by the model. The result of 0.9914 shows that the model accounts for 99.14% of the price variance on the test set, where 1.0 represents a perfect fit.

Hit rate by tolerance threshold The hit rate is the proportion of predictions where the error falls within a specified threshold, k :

$$\text{HitRate}(k) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[|\hat{p}_i - p_i| \leq k] \quad (18)$$

The evaluation yielded the following distribution:

Table 1: Hit rate by tolerance threshold

Threshold k	Proportion	Interpretation
≤ 1 pip	73.49%	Over 73% of predictions are within one pip
≤ 3 pips	93.68%	Over 93% of predictions are within three pips
≤ 5 pips	99.53%	Nearly all predictions are within five pips
≤ 10 pips	99.96%	Virtually all predictions fall within ten pips

Average bias The average bias is defined as:

$$\text{Bias} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - p_i) \quad (19)$$

The calculated average bias is -0.71 pips. This negative value indicates that the model has a slight tendency to underestimate the actual prices. This behavior arises from the distribution of price variations in the historical data, combined with the loss function minimization process.

Direction accuracy The direction accuracy measures the percentage of correct predictions regarding the upward or downward movement of the exchange rate:

$$\text{Dir. Acc.} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\text{sign}(\hat{\delta}_i) = \text{sign}(\delta_i)] \quad (20)$$

The direction accuracy is 22.05%. This percentage is lower than fifty percent, which would represent a random guess. This low rate is a direct consequence of the systematic negative bias of -0.71 pips. Because the model frequently predicts a negative delta to minimize the huber loss, it tends to project downward movements, reducing its capacity to correctly identify upward price movements.

8 Conclusion and future work

8.1 Project summary

This project has successfully implemented and deployed a real time prediction system for the EUR/USD exchange rate. The main contributions include:

1. A hybrid architecture combining a bigru and a multi head attention mechanism with positional encoding, trained on a five year historical dataset of EUR/USD rates.
2. A variation based prediction method using standard scaling, which eliminates the level bias of +47 pips observed when predicting absolute prices directly.
3. A complete streaming pipeline integrating the Yahoo Finance api, Apache Kafka, and the bigru model for real time predictions.
4. Documented performance results showing a mean absolute error of 1.18 pips, a root mean square error of 1.59 pips, a coefficient of determination of 0.9914, and 93.68% of predictions within a tolerance of three pips.

8.2 System limitations

- The prediction model is static, meaning it does not update its parameters online to adapt to shifting market environments.
- The low direction accuracy, which is restricted by the model learning a systematic bias, limits the utility of the current system for high frequency directional trading strategies.
- TensorFlow versions starting from 2.11 do not support native GPU acceleration on Windows, which restricts training and inference speed unless a virtualized environment is used.
- The Yahoo Finance api limits high frequency minute by minute data history to eight days, restricting the size of the test dataset.

8.3 Future work

- Integrate online learning techniques to update model weights incrementally as new streaming data arrives.
- Incorporate technical indicators such as the relative strength index or bollinger bands, alongside macroeconomic announcements, to enrich the input features.

- Expand the streaming pipeline to support multiple currency pairs, including the GBP/USD and USD/JPY exchange rates.
- Connect the prediction outputs to a risk management module for automated order execution.